

Introducción a Ciencias de la Computación Tarea 1

Aarón Palafox Solís

29 de Septiembre

Profesor: Salvador López Mendoza

Ayudante: Yanahí Demerio Torres

Ayudante de Laboratorio: Rosa Victoria Villa Padilla

Fecha de entrega: 5 de Octubre a las 23:59pm

Problema 1: Aplicación de entrega de comida

Una aplicación permite que los clientes soliciten comida a diferentes restaurantes.

El cliente selecciona un restaurante y realiza un pedido. Un pedido puede contener varios productos del restaurante. Cada producto tiene un nombre, un precio y puede tener diferentes opciones de personalización.

Después de confirmar el pedido, el sistema calcula el total y registra el pago. Posteriormente, un repartidor acepta el pedido y lo recoge en el restaurante para entregarlo al cliente.

El sistema debe permitir conocer el estado del pedido. Por ejemplo, un pedido puede pasar por los siguientes estados:

Pendiente → Preparando → Listo → En camino → Entregado

• **Tabla de clases, objetos, atributos y métodos**

Objetos	Clases	Atributos	Métodos
cliente	Cientes	nombre dirección	identificarCliente seleccionarmetodoPago personalizarProducto escribirDirección seleccionarRest seleccionarProd realizarPedido seleccionarcantProd realizarPago
datosCliente	Datos	información	registrarDatoscliente enviarRestaurante
restaurante	Restaurantes	nombre horarioServicio	recibirPedido actualizarEstado entregarRepartidor
producto	Productos	nombre precio opcionesPersonalización	consultarProd
pedido	Pedido	total estado fecha restaurante repartidor cliente productos cantidadselecProd dirección	confirmarPedido calcularTotal recibirEstado agregarProd asignarRep generarTicket cambiarEstado
pago	Pago	monto metodoPago fecha estadoPago	confirmarPago registrarPago mostrarMonto recibirTotal
repartidor	Repartidor	nombre	aceptarPedido recogerPedido entregarPedido

•Determinar las responsabilidades

-Cliente: identificarse, seleccionarRestaurante, seleccionarProducto, realizarPedido, personalizarProducto seleccionarmetodoPago y realizarPago.

-Restaurante: recibirPedido y actualizarEstado.

-Producto: consultarProducto.

-Pedido: confirmarPedido, calcularTotal, recibirEstado, agregarProducto, asignarReparti-

dor, cambiarEstado y generarTicket.

-Pago: recibirTotal, mostrarMonto, confirmarPago y registrarPago.

-Repartidor: aceptarPedido, recogerPedido y entregarPedido.

•Describir la interacción entre objetos

El objeto "cliente" recibe el nombre de la persona que realizará el pedido y lo guarda en la variable nombre y también se recibe la dirección. También desde este objeto se selecciona un restaurante y se seleccionan los productos y la cantidad de estos del restaurante seleccionado, se realiza el pedido y se realiza el pago. Además de poder personalizar el producto.

En el objeto "datoscliente" se almacena la información que el cliente proporcionó, su nombre y dirección; envía la información al restaurante para fines estadísticos.

En el objeto "restaurante" recibe el pedido realizado por el cliente y actualiza el estado del pedido de pendiente a preparando.

En el objeto "producto" se consulta información de los productos que el cliente seleccionó.

En el objeto "pedido" se calcula el total a pagar dependiendo de la cantidad, el tipo y el precio de los productos. También se van almacenando los cambios de estado. Se guarda la fecha de realización del pedido, se guarda el nombre del restaurante seleccionado y el nombre que el cliente escribió, así como el producto seleccionado y la cantidad. Se asigna un repartidor y por último se genera un ticket con toda la información del pedido.

En el objeto "pago" de pedido se muestra un monto que proviene de pedido, cuando el cliente realiza el pago y selecciona su método de pago, aquí se confirma si fue rechazado o aceptado. Una vez aceptado, se registra el pago con una fecha y el monto pagado.

Por último el objeto "repartidor" recibe toda la información del pedido, recoge el pedido en el restaurante, el cuál actualiza el estado del pedido a "en camino" y entrega el pedido, donde se actualiza el estado del pedido a "entregado".

•Definir los escenarios

Escenario 1:

1. El cliente escribe su nombre y dirección.
2. El cliente selecciona un restaurante.
3. El cliente selecciona y personaliza los productos que quiere y la cantidad.
4. El cliente realiza el pedido.
5. Pedido genera un ticket con la información del pedido.
6. Pago recibe el total.
7. Pago muestra monto.
8. El cliente elige un método de pago.
9. El cliente realiza el pago.

10. Pago confirma el pago. El pago es rechazado.

Escenario 2:

1. El cliente escribe su nombre y dirección.
2. El cliente selecciona un restaurante.
3. El cliente selecciona y personaliza los productos que quiere y la cantidad.
4. El cliente realiza el pedido.
5. Pedido genera un ticket con la información del pedido.
6. Pago recibe total.
7. Pago muestra el monto.
8. El cliente selecciona un método de pago.
9. El cliente realiza el pago.
10. Pago confirma el pago. El pago es aceptado
11. Pago registra el pago.
12. Pedido confirma el pedido.
13. Pedido pone estado inicial "pendiente".
14. Repartidor acepta pedido.
15. Pedido asigna repartidor.
16. Restaurante recibe pedido.
17. Pedido pone estado en "preparando".
18. Restaurante actualiza el estado.
19. Pedido pone estado en "listo".
20. Restaurante actualiza estado.
21. Repartidor recibe pedido
22. Pedido pone estado en "en camino".
23. Restaurante actualiza el estado.
24. Repartidor entrega pedido y estado se actualiza a "entregado".

Pregunta de análisis

El pedido tiene diferentes estados:

Pendiente, Preparando, Listo, En camino, Entregado

¿Considerarías que el estado simplemente como un atributo de la clase **Pedido**, o utilizarías otra clase para representarlo?

Lo consideraría como un atributo de la clase Pedido.

Explica y justifica tu decisión.

Porque a mi parecer funciona más como una característica, que va cambiando dependiendo de las acciones que se van realizando a lo largo del proceso, además que simplifica la estructura del código. Ya que, si lo defino como una clase, tendría que crear objetos "pedido" con cada uno de los objetos "estado" aplicando el método "actualizarEstado" lo que es menos práctico desde mi punto de vista.

Problema 2: Sistema de reservaciones de un hotel

Un hotel desea desarrollar un sistema para administrar las reservaciones de sus habitaciones.

El hotel cuenta con diferentes tipos de habitación, por ejemplo:

- Individual
- Doble
- Suite

Cada habitación, tiene un número, una capacidad y un precio por noche.

Un cliente puede realizar una reservación indicando las fechas de entrada y salida . El sistema debe verificar que exista una habitación disponible para esas fechas.

Una reservación puede incluir servicios adicionales, como desayuno, estacionamiento o servicio a la habitación. Cada servicio tiene un costo.

Al finalizar la reservación, el sistema debe calcular el costo total. Cuando el cliente realiza el pago, la reservación queda confirmada.

·**Tabla de clases, objetos, atributos y métodos**

Objetos	Clases	Atributos	Métodos
hotel	Hotel	nombre numeroHabitaciones	verificarDisponibilidad
habitacion	Habitaciones	numeroHabitacion capacidad precioNoche	individual doble suite consultarPrecio
servicios	ServicioAdicional	nombre precio	obtenerNombre obtenerPrecio
reservación	Reservaciones	habitación fechaEntrada fechaSalida servicios nombreCliente	agregarHab agregarEntrada agregarSalida agregarServicio calcularCostoServicios calcularTotal confirmarReservación
cliente	Cliente	nombre	registrarNombre realizarReservación seleccionarHabitación seleccionarServicios seleccionarMetodoPago realizarPago registrarEntrada registrarSalida
pago	Pago	monto estadoPago metodoPago fecha	cobrarTotal confirmarPago registrarPago generarTicket mostrarMonto

•Determinar las responsabilidades

-Cliente: registrarEntrada, registrarSalida, registrarNombre, realizarReservación, seleccionarHabitación, seleccionarServicios, seleccionarMetodoPago y realizarPago.

-Reservación: agregarHabitación, agregarEntrada, agregarSalida, agregarServicio, calcularCostoServicios, calcularTotal y confirmarReservacion.

-Hotel: verificarDisponibilidad.

-Habitación: consultarPrecio.

-Pago: mostrarMonto, recibirTotal, confirmarPago, registrarPago y generarTicket.

-Servicios: obtenerNombre y obtenerPrecio.

•Describir la interacción entre objetos

El objeto "cliente" recibe el nombre de la persona que hará la reservación, así como desde aquí el cliente registra el día que quiere entrar y salir. Se selecciona una habitación y los servicios adicionales que quiera agregar. Se selecciona un método de pago y se realiza el pago.

El objeto "hotel" se verifica la disponibilidad de la habitación que haya escogido el cliente.

En el objeto "habitación" se consulta el precio de la habitación escogida por el cliente.

En el objeto "reservación" se recibe el tipo de habitación que se haya escogido, se agrega el día de entrada y salida. También se agregan los servicios adicionales que haya solicitado el cliente se calcula el costo total de los servicios añadidos, se calcula el total del costo de la habitación más los servicios añadidos. Por último se confirma el pedido.

En el objeto "pago" se recibe el total calculado de la reservación, se le muestra el monto al cliente, cuando el cliente realiza el pago, se confirma el pago, se registra el pago y se genera un ticket de compra.

•Definir los escenarios

Escenario 1:

- 1.El cliente proporciona su nombre.
2. El cliente selecciona día de entrada.
3. El cliente selecciona día de salida.
4. El cliente selecciona una habitación.
5. El sistema valida la disponibilidad de la habitación.
6. El cliente selecciona los servicios adicionales que desee
7. Reservación calcula los costos de los servicios.
8. Reservación calcula los costos totales.
9. Pago muestra el monto a pagar.
10. El cliente realiza el pago.
11. Pago registra el pago del cliente.
12. Se confirma la reservación.
13. Se genera ticket de compra.

Pregunta de análisis

En el problema aparecen los tipos de habitación:

Individual, Doble y Suite

¿Crearías una clase diferente para cada tipo de habitación?

Por ejemplo :

HabitaciónIndividual

HabitaciónDoble

HabitaciónSuite

Respuesta: No, crearía una sola clase llamada habitación

Explica y justifica tu desición.

Porque con la clase habitación, solamente creo métodos constructores de la clase y ya solo uso un objeto y solo modifíco los atributos de este. Así se simplifica al momento de hacer el código y a mi parecer es más práctico que crear tres clases y objetos diferentes.

Problema 3: Sistema para administrar un torneo de videojuegos

Se desea desarrollar un sistema para administrar un torneo de videojuegos.

En el torneo participan varios jugadores. Cada jugador tiene un nombre y un identificador. Los jugadores pueden pertenecer a diferentes equipos.

El torneo se divide en partidas. En cada partida pueden participar dos jugadores o dos equipos, dependiendo de la modalidad del torneo.

Cada partida registra su resultado y determina qué participante avanza a la siguiente ronda.

El torneo puede tener diferentes modalidades de juego y cada modalidad puede utilizar reglas diferentes para determinar al ganador.

El sistema debe permitir:

- Registrar jugadores.
- Crear equipos.
- Registrar partidas.
- Registrar resultados.
- Determinar quién gana cada partida.
- Determinar qué participantes avanzan a la siguiente ronda.

·Tabla de clases, objetos, atributos y métodos

Objetos	Clases	Atributos	Métodos
jugador	Jugadores	nombre equipo	ingresarNombre ingresarEquipo asignarId seleccionarModalidad seleccionarNumPartidas
modalidad	Modalidad	jugador1 jugador2 equipo1 reglas equipo2	jugadores equipos obtenerJugadores obtenerEquipos
partida	Partidas	numPartidas jugador1 jugador2 equipo1 equipo2 numeroPartida	asignarNumPartidas asignarJugadores asignarEquipos avanzarJugador avanzarEquipo registrarResultado registrarPartida
torneo	Torneo	jugadorAvanza equipoAvanza jugadores equipos partidaGanador IdJugador	asignarJugadorAvanza asignarEquipoAvanza registrarJugadorAvanza registrarEquipoAvanza asignarId establecerReglas
resultado	Resultado	numeroPartida ganadorPartida	asignarResultado mostrarResultado asignarPartida

•Determinar las responsabilidades

-Jugador: ingresarNombre, ingresarEquipo, seleccionarNumPartidas y seleccionarModalidad

-Modalidad: obtenerJugadores y obtenerEquipos.

-Partida: asignarNumPartidas, asignarJugadores, asignarEquipos, avanzarJugador, avanzarEquipo, registrarPartida y registrarResultado.

-Torneo: asignarId, asignarJugadorAvanza, asignarEquipoAvanza, registrarJugadorAvanza, establecerReglas y registrarEquipoAvanza.

-Resultado: asignarPartida, asignarResultado y mostrarResultado

•Describir la interacción entre objetos

El objeto "jugador" recibe el nombre del jugador que se inscriba, así como el equipo al que pertenece el jugador y se selecciona la modalidad. Además de seleccionar cuántas partidas

quiere jugar.

El objeto "modalidad" si el jugador selecciona la modalidad de jugadores el objeto obtiene el nombre del jugador1 y el del jugador2. En caso de seleccionar equipo, obtiene el nombre del equipo1 y del equipo2

En el objeto "partida" asigna el numero de partidas seleccionadas por el jugador, asigna los nombres de los jugadores en jugador1 y jugador2, o en otro caso el nombre del equipo1 y equipo2. Al terminar la partida determina que equipo o jugador avanza y registrar el número de partida que se jugó.

En el objeto "torneo" se asigna un id al jugador cuando se registra, asigna el nombre del jugador o equipo que avanza, registra el nombre del jugador o equipo que avanzó y establece las reglas de cada modalidad.

En el objeto "resultado" se asigna qué jugador o equipo ganó por partida y el se asigna el número de partida. Para al final mostrar el resultado ppor partida.

•Definir los escenarios

Escenario 1:

1. Jugador ingresa su nombre.
2. El sistema registra el nombre del jugador.
3. El sistema le asigna un ID al jugador.
4. El sistema registra el nombre de su equipo.
5. El jugador selecciona la modalidad.
6. El jugador escoge la cantidad de partidas que quiere jugar.
7. El sistema asigna un contrincante.
8. La partida finaliza
9. El sistema registra ganador.
10. El sistema registra numero de ronda.
11. El sistema muestra los ganadores por ronda.

Pregunta de análisis

1. Cada partida tiene un resultado.

¿Considerarías que el resultado es simplemente un atributo de la clase Partida, por ejemplo:

resultado = "Jugador 1"

¿O podría ser conveniente crear una clase Resultado?

Respuesta: Creé una clase resultado porque los atributos varían según la modalidad que escoja el jugador.

Explica en qué situación tendría sentido cada alternativa y cuál utilizarías en este problema.

Respuesta: Creo que tendría sentido verlo como atributo si el resultado estuviera únicamente en termino de jugador1 o jugador2 y no se pudiera escoger una modalidad en equipos.

La alternativa de verlo como clase es conveniente en este caso porque los términos que se van a usar para representarlo, dependen la modalidad que el jugador escoja.

2. El problema indica que existen diferentes modalidades de juego y que cada modalidad puede utilizar reglas diferentes para determinar al ganador.

¿Representarías la modalidad únicamente mediante un atributo de Partida, por ejemplo:

modalidad = "Individual"

¿O considerarías conveniente crear una clase que represente una Modalidad?

Respuesta: Lo consideraría como una clase.

Explica y justifica tu decisión.

Porque las reglas varían según la modalidad entonces hago unos constructores "equipos" y "jugadores" y ahí establezco las reglas, porque de otra manera tendría que crear clases diferentes para cada modalidad, lo que desde mi punto de vista no es tan práctico.